

RideNow: A Ride-Sharing Trip Manager

Object-Oriented Programming — Lab 3

Marks: 10 **Language:** Java (JDK 17) **Approach:** Test-Driven

Submit your .java source files only. Do not modify the test file.

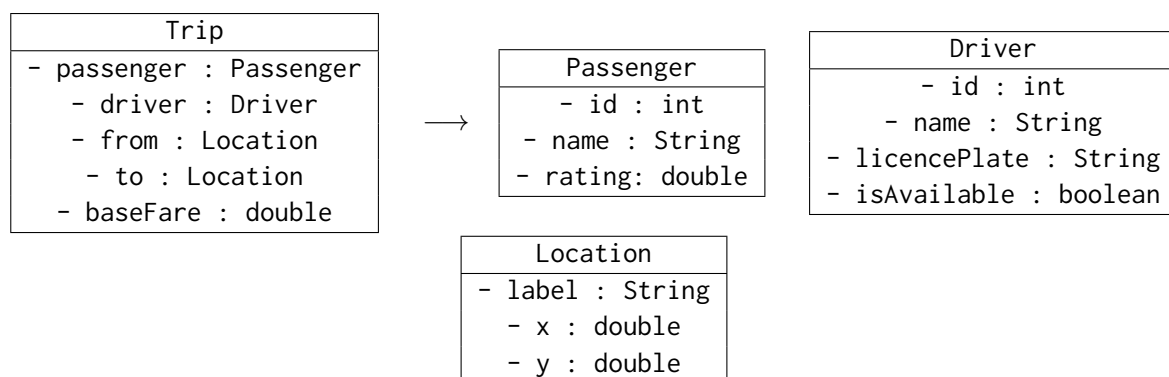
What this lab teaches. Classes can *hold other classes* as fields — this is called **composition**. You will also practice writing **constructors** (including classes with more than one constructor), and working with Java's common **data types** (int, double, String, boolean). The final part challenges you to connect everything together.

The Scenario

RideNow is a ride-sharing app. When a passenger books a ride, the system needs to record useful information about the trip: who the passenger is, which driver picks them up, where the trip starts and ends, and what the fare will be.

In this lab you will build the four classes that power a single trip: **Location**, **Passenger**, **Driver**, and **Trip**.

The diagram below shows how the classes relate to one another. Trip *owns* a Passenger, a Driver, and two Location objects — this is **class composition** in action.



Part A — Location [2 marks]

Location represents a named point on the city map.

Fields (all private)

- String label — a human-readable name (e.g. "Airport")
- double x — east coordinate in kilometres
- double y — north coordinate in kilometres

Constructors

- `Location(String label, double x, double y)` — full constructor.
- `Location(double x, double y)` — convenience constructor; sets label to "Unknown" automatically.

Methods

- `getLabel()`, `getX()`, `getY()` — standard getters.
- `double distanceTo(Location other)` — Euclidean straight-line distance: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.
- `String toString()` — returns e.g. "Airport (3.00, 7.50)".

Part B — Passenger [2 marks]

Passenger represents a customer who books rides.

Fields (all private)

- `int id`, `String name`, `double rating` (a number from 1.0 to 5.0).

Constructors

- `Passenger(int id, String name, double rating)` — full constructor.
- `Passenger(int id, String name)` — new passengers start with a default rating of 5.0.

Methods

- `getId()`, `getName()`, `getRating()` — getters.
- `void updateRating(double newRating)` — replaces the stored rating.
- `String toString()` — returns e.g. "Passenger[42] Alice (4.80)".

Part C — Driver [2 marks]

Driver represents a registered RideNow driver.

Fields (all private)

- `int id`, `String name`, `String licencePlate`, `boolean isAvailable`.

Constructors

- `Driver(int id, String name, String licencePlate)` — a newly registered driver is *available* by default.
- `Driver(int id, String name, String licencePlate, boolean isAvailable)` — full constructor, sets availability explicitly.

Methods

- `getId()`, `getName()`, `getLicencePlate()`, `isAvailable()` — getters.
- `void setAvailable(boolean available)` — updates availability.
- `String toString()` — returns e.g. "Driver[7] Bob (DHA-1234) [AVAILABLE]" or "[BUSY]".

Part D — Trip [4 marks]

Trip is the most important class. It *composes* the three classes above: every trip holds a Passenger, a Driver, and two Location objects (pick-up and drop-off).

Key idea. A Trip object *contains* other objects as fields. When `trip.getDistance()` method is called, the Trip asks its own from and to fields to compute the distance — one object delegating work to another.

Fields (all private)

- Passenger passenger
- Driver driver
- Location from
- Location to
- double baseFare — a fixed per-trip charge in BDT.
- boolean completed — starts as false.

Constructor

- `Trip(Passenger passenger, Driver driver, Location from, Location to, double baseFare)` — stores all five arguments; sets `completed = false` and marks the driver as unavailable (calls `driver.setAvailable(false)`).

Methods

- **Getters:** `getPassenger()`, `getDriver()`, `getFrom()`, `getTo()`, `getBaseFare()`, `isCompleted()`.
- `double getDistance()` — returns the straight-line distance between from and to (delegate to Location).
- `double calculateFare()` — total fare in BDT:

$$\text{fare} = \text{baseFare} + 15.0 \times \text{distance}$$

where 15.0 is the per-kilometre rate.

- `void completeTrip()` — marks the trip as completed and marks the driver as available again (`driver.setAvailable(true)`).
- `String getSummary()` — returns a multi-line summary string in this exact format (values are examples):

```
Trip Summary
  Passenger : Alice
   Driver   : Bob (DHA-1234)
    From     : Airport (3.00, 7.50)
```

```

To      : University (8.00, 4.00)
Distance : 6.10 km
Fare     : BDT 141.50
Status   : COMPLETED

```

Status is COMPLETED when completed is true, otherwise IN PROGRESS.

Marking Rubric

Part	Criterion	Marks
A	Location tests pass	2
B	Passenger tests pass	2
C	Driver tests pass	2
D	Trip tests pass (composition, delegation, fare)	3
All	All fields private; multiple constructors used correctly	1
Total		10

General Notes

- Use double for all coordinates, distances, fares, and ratings.
- Use `Math.sqrt` from `java.lang.Math` for distances.
- For formatted output use `String.format("%.2f", value)`.
- Do *not* import `ArrayList` or any collection — you will not need them.
- Compile with `javac` on JDK 17; run with your IDE or `mvn test`.

“A class is not just a bundle of code. It is a model of something in the real world.”